SANAD: Zero-PII Verifiable Credentials for Multilingual Government-Service Navigation in the UAE

Sanmeet Singh Kohli[1], Maria Mustafa[1], Sakshi Himpalnerkar[2], and Haitham Yaish[1]

[1] University of Wollongong in Dubai, Dubai, United Arab Emirates
[2] Maharashtra Institute of Technology, Chhatrapati Sambhajinagar, India

Abstract. The UAE is among the world's leading digital governments, yet its largely expatriate residents face three barriers: cross-language navigation, understanding which documents and rules apply, and producing privacy-safe records of unresolved complaints. We present SANAD, a prototype assistant. It combines a schema-constrained large-language-model front end, which maps free-text situations only to services in a curated catalogue of official UAE portals, with a zero-personally-identifiable-information (zero-PII) credential mechanism that anchors each user-confirmed claim as a salted SHA-256 digest recorded by an issuer-controlled smart contract on the Polygon Amoy testnet. An honest-degradation policy labels every AI output by source and blocks sample fallbacks from credential issuance. Zero-PII scopes to the public anchor; application data remains personal and access-controlled. The chain attests only a record's existence and immutability, never its truth. Twelve automated tests pass, including live-PostgreSQL row-level-security isolation, and a 16-case grounded-matcher benchmark returns 16/16 acceptable top matches with every delivered identifier catalogue-valid.

Keywords: electronic government, verifiable credentials, blockchain, large language models, United Arab Emirates

1 Introduction

The United Arab Emirates consistently performs at the top of global digital-government assessments: in the 2024 United Nations E-Government Survey it ranks 11th of 193 countries, with an E-Government Development Index score of 0.9533 [1], and the UAE government reports first place worldwide in the E-Government Literacy sub-index [2]. Availability of online services, however, is not the same as accessibility of outcomes. The UAE's population is majority expatriate on the government's own account [11], a figure commonly cited at roughly 88%, and residents routinely interact with services such as labour complaints, visa and status questions, wage protection, unemployment insurance and tenancy disputes, whose entry points are spread across portals such as u.ae and the Ministry of Human Resources and Emiratisation (MOHRE) [10].

Three gaps at the point of use motivate the design. First, a language gap: official information exists in several languages, but guidance quality and user confidence can diverge for residents whose strongest language is Hindi, Urdu or Bengali rather than Arabic or English. Second, an evidence and eligibility gap: residents often cannot tell which document proves what, which scheme they qualify for, or what a rejected application means. Third, a trust gap: when a dispute such as unpaid wages escalates, the worker's own records are informal, scattered and unverifiable to a third party, while any centralised attempt to build verifiable records collides with the UAE Federal Decree-Law No. 45 of 2021 on the Protection of Personal Data (PDPL) [7].

This paper describes SANAD, a working prototype that addresses the three gaps through one conservative design decision: personal data never leaves the user's own database partition, and the only thing written to a public blockchain is a salted cryptographic digest of a user-confirmed claim. SANAD does not detect false claims, and it does not assert that anything on-chain is factually true. It provides existence, immutability and timestamping proofs for records whose accuracy the user confirms.

Contributions. (1) A grounded navigation pipeline in which a large language model (LLM) may only select service identifiers from a curated catalogue of official portals, so that out-of-catalogue services cannot reach the user; an incorrect match to a valid catalogue entry nevertheless remains possible and is treated as an open risk (Sect. 6.1). (2) A zero-PII verifiable-credential mechanism: canonical serialisation, a 32-byte random salt and a domain-separated SHA-256 digest, anchored by an issuer-controlled Solidity registry, with verification that recomputes integrity off-chain and existence on-chain. (3) An honest-degradation policy for AI features: sample fallbacks are clearly labelled, carry zero confidence and are blocked from credential issuance. (4) An automated validation suite, including row-level-security (RLS) tests against a live PostgreSQL instance, together with a published taxonomy of what is and is not yet proven (Sect. 6, Table 2).

Section 2 positions the work against prior art; Sect. 3 presents the architecture; Sect. 4 formalises the credential mechanism; Sect. 5 describes the grounded language layer; Sect. 6 reports implementation and validation evidence; Sect. 7 discusses limitations and ethics; Sect. 8 concludes.

2 Related Work

2.1 Blockchain-Backed Credentials

Since Nakamoto's proposal of an append-only, trust-minimised ledger [9], credential anchoring has become a recurring application: Blockcerts defined an open standard for blockchain-verified academic credentials issued by institutions [4], and later work examined blockchain approaches specifically for academic credentials beyond the issuing institution [5]. The W3C Verifiable Credentials Data Model gave the field a common representation for claims, proofs and subjects [3]. These systems typically put structured credential content, often including identifiers of the holder, into a signed artifact whose validity is checked against a ledger. SANAD takes the more restrictive route: no credential content is published at all. Only a digest computed over

a canonicalised snapshot with a secret salt is recorded, so the design is intended to prevent direct disclosure of claim contents through the public artifact and to resist dictionary enumeration of claims; metadata-level caveats are given in Sect. 4.4.

2.2 Language Access and LLM-Grounded Public Services

Recent surveys of code-switched and multilingual natural-language processing emphasise that monolingual assumptions systematically degrade service quality for South Asian language communities, the same groups that dominate the UAE's expatriate workforce [6]. LLM assistants can lower the language barrier, but unconstrained generative systems are unsuitable for public-service guidance because they can invent plausible but non-existent schemes. SANAD constrains generation at the schema level: the model returns only catalogue identifiers and short rationales, and every identifier is joined back to a curated entry whose URL points to a government domain. Accessibility is treated as a first-class requirement following WCAG 2.2 [8], including right-to-left layouts for Arabic and Urdu.

2.3 Positioning

Existing gov-tech assistants optimise for conversational breadth; existing blockchain-credential systems optimise for portability of institutional attestations. SANAD targets the intersection that matters for resident-level disputes: a multilingual intake whose outputs are constrained to a curated catalogue, and a trust artifact whose public form contains no personally identifiable information (PII), which reduces, though does not eliminate, consent risk (Sect. 4.4). The PDPL principles of data minimisation and purpose limitation [7] guide the design: PII is confined to RLS-protected tables, and the public artifact is a hash. Table 1 compares the approaches across the four properties that matter for this use case.

Table 1. How SANAD differs from closely related approaches across the four properties that matter for resident-level, privacy-sensitive service navigation.

Approach	PII / content on-chain	Grounded to official catalogue	Honest-degradation policy	User-confirmed anchor
Blockcerts / institutional VC [3, 4]	Signed credential content	No	No	Issuer attestation
Blockchain academic credentials [5]	Credential identifiers	No	No	Issuer attestation
Generic LLM gov-assistant	None, but no anchor	No (hallucination risk)	No (silent fallbacks)	None
SANAD (ours)	None (salted digest only)	Yes (28-entry catalogue)	Yes (labelled, 503)	Yes (user-confirmed)

3 System Architecture

SANAD is a full-stack web application: a Next.js 16 (App Router) front end with TypeScript in strict mode, server route handlers as the only trusted API boundary, Supabase (PostgreSQL with row-level security and the @supabase/ssr session helper) for persistence, Gemini accessed through the Vercel AI SDK's generateObject call with a Zod schema, and ethers v6 for transaction signing against the SANADCredential contract on Polygon Amoy (chain ID 80002). Figure 1 shows the four layers and the flows between them.

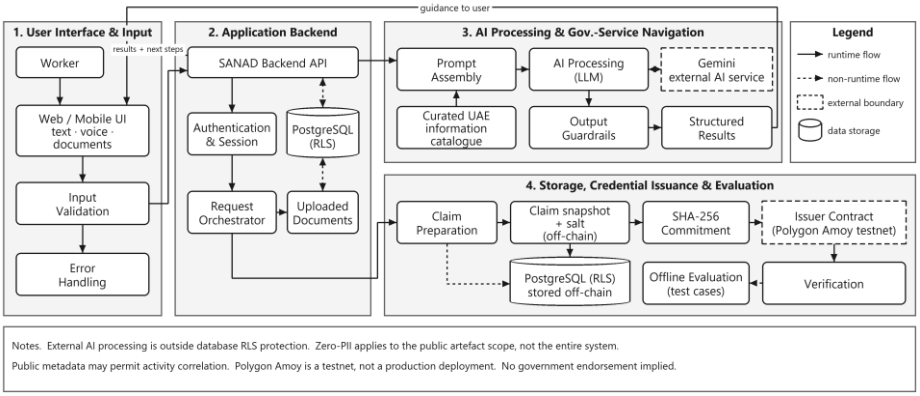


Fig. 1. System architecture in four layers: (1) user interface and input; (2) application backend with RLS-protected PostgreSQL; (3) AI processing grounded in the curated catalogue, with Gemini as an external service; (4) storage, credential issuance and offline evaluation. The claim snapshot and salt stay off-chain. Only an opaque identifier and a SHA-256 commitment reach the testnet contract, which attests existence and revocation status, never content or truth.

A session begins either with Supabase authentication or with a signed demo cookie (HMAC-SHA-256 over an identifier and expiry, httpOnly, SameSite=Lax). Every mutating route handler resolves the caller through a single actor() helper before touching data; unauthenticated requests receive 401 and the interface prompts for sign-in rather than silently showing another user's data. The user describes a situation in their own language; the AI layer extracts structured facts and matches the situation against the official-services catalogue; document analysis (e.g. a wage slip or contract photo) can attach evidence summaries; the user then confirms or edits the resulting claim statement. Only a confirmed claim is snapshotted, hashed and, in real mode, issued on-chain. The catalogue itself is data, not model memory: 28 curated entries, each linking to u.ae or mohre.gov.ae, with supported-situation tags used for deterministic pre-filtering before the model is consulted.

The interface ships with five hand-curated locales: English, Arabic and Urdu (both right-to-left), Hindi and Bengali. An in-page machine translator covers further resident languages, so the intake path works without English proficiency. Figure 2 shows four production screens captured from the running prototype.

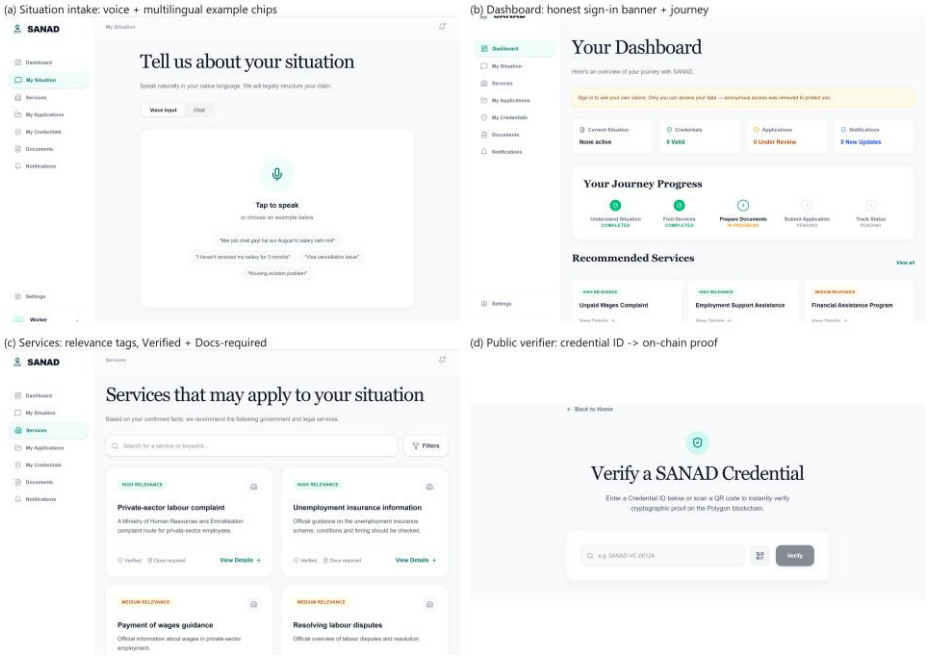


Fig. 2. Prototype screens (captured live, not mock-ups). (a) Situation intake with voice input and multilingual example prompts; (b) dashboard with the explicit sign-in banner that replaced anonymous claim access and the five-stage journey tracker; (c) grounded service list with relevance, Verified and Docs-required tags linking to official portals; (d) the public verifier that resolves a credential ID to an on-chain existence proof.

4 The Zero-PII Credential Mechanism

4.1 Canonicalisation and Salted Digest

When a user confirms a claim, the server builds an immutable snapshot (statement, extracted facts, provenance flag USER_CONFIRMED), serialises it with a canonical encoder that sorts object keys, recursively normalises arrays, JSON-escapes strings and leaves numbers, booleans and null in their JSON forms, and computes

$$h = SHA\text{-}256(\text{“SANAD:v1:”} \,\|\, salt \,\|\, \text{“:”} \,\|\, canonical(snapshot))$$

where salt is 32 fresh random bytes per credential. The domain-separator prefix (“SANAD:v1:”) prevents cross-protocol collisions; the salt makes the digest non-malleable and non-dictionary-matchable, since an observer who guesses the snapshot contents still cannot reproduce h without the salt. Both snapshot and salt are stored only in RLS-protected off-chain tables; the pair (credential identifier, h) is what crosses the trust boundary. A UUID credential identifier is mapped to a bytes32 by left-padding its hex digits (SHA-256 only for non-hex identifiers), so the on-chain identifier is deliberately linkable to its credential record while revealing no personal

data: the salt protects the snapshot digest, not the identifier. Integrity recomputation runs server-side through the same code path; we do not specify a standalone encoding standard, so byte-for-byte reproduction by a third-party verifier is not guaranteed.

4.2 Issuer-Controlled Registry

The on-chain component is deliberately minimal: a single contract whose issuer key is immutable, with issue, revoke and a free status view. Listing 1 is an abridged excerpt: the constructor, the Revoked event and the revoke body (guarding NotIssued and AlreadyRevoked) are omitted for space but implement exactly the semantics described here. Duplicate issuance for an identifier is rejected; revocation is one-way; the stored record hash is never overwritten, giving tamper-evidence against any later change of the off-chain snapshot. The issuer account is the project team's deployment wallet, not a government authority: an anchor therefore records the existence of a user-confirmed claim and must not be read as official government verification.

```
contract SANADCredential {
    address public immutable issuer;
    struct Credential { bytes32 recordHash; address issuedBy;
                        uint64 issuedAt; bool revoked; }
    mapping(bytes32 => Credential) private records;
    event Issued(bytes32 indexed id, bytes32 indexed recordHash,
                 address indexed issuer);
    modifier onlyIssuer() {
        if (msg.sender != issuer) revert NotIssuer(); _; }
    function issue(bytes32 id, bytes32 recordHash)
            external onlyIssuer {
        if (records[id].issuedAt != 0) revert AlreadyIssued();
        records[id] = Credential(recordHash, msg.sender,
                                 uint64(block.timestamp), false);
        emit Issued(id, recordHash, msg.sender);
    }
```

```
    function status(bytes32 id) external view
            returns (bytes32, address, uint64, bool) {
        Credential storage e = records[id];
        return (e.recordHash, e.issuedBy, e.issuedAt, e.revoked);
    }
}
```

Listing 1. Abridged excerpt of the SANADCredential registry (Solidity ^0.8.24, MIT); complete source is in the repository.

4.3 Verification Semantics and What They Do Not Prove

Verification composes two independent checks. Integrity: recompute h from the stored snapshot and salt and compare with the stored record hash. Existence: read status(id) on the chain and confirm the returned hash matches, the timestamp is present and the revoked flag is false. A credential is VALID only if both hold. The chain attests only that a given digest existed at a given time and has not been altered. It cannot attest that the underlying statement about the world is true. SANAD encodes this limit in the product itself: verification pages and issuance dialogs state the distinction, and the paper throughout uses “verifiable” only in the cryptographic sense.

4.4 Threat Model, Issuer-Key Risk and Retention

We make three adversary classes explicit. A chain observer sees only an opaque identifier and a 32-byte digest; because the digest is salted with a per-credential 32-byte secret held off-chain, guessing the snapshot contents is insufficient to reproduce it, so the observer cannot recover claim content through a dictionary attack. A database adversary who defeats row-level security can read snapshots and salts, but still cannot forge a valid on-chain anchor without the issuer key, and any silent edit of a stored snapshot breaks the integrity recomputation in Sect. 4.3. A holder of the issuer key can issue or revoke records; this is the principal trust assumption, and we do not claim to remove it. The issuer address is immutable and every issue or revoke emits a public event, so abuse stays possible but is visible and attributable. Production use should replace the single key with a multisig or HSM and a documented revocation policy.

Retention and the zero-PII scope are stated narrowly. Snapshots and salts persist server-side under RLS. User-facing deletion is currently implemented for uploaded documents; claim-level deletion of snapshots and salts is a designed control that the prototype does not yet ship. Deletion would remove nothing from the chain, where digests and events are immutable. It withdraws the data the application needs to recompute the digest, so the anchor stays on-chain but can no longer be verified. This is the intended right-to-erasure behaviour once deletion ships: an anchor cannot be verified after the consent behind it is withdrawn. Finally, “zero-PII” is a statement about the public artifact, not a legal conclusion: the design is consistent with the

PDPL principles of data minimisation and purpose limitation [7], but we do not claim that it, by itself, establishes regulatory compliance.

The salted digest does not hide everything. Public transaction metadata (the issuer address, event ordering and block timestamps) can correlate issuance activity over time even though it reveals no claim content, and application logs, session identifiers and the anchoring wallet address are themselves potentially personal data under the PDPL; “zero-PII” therefore describes the public credential artifact, not the system's entire data footprint. Separately, situation text and documents submitted for analysis are transmitted to the model provider (Google's Gemini API, reached through the AI SDK) for inference; while in flight or in the provider's systems they are outside our database and therefore outside RLS's protection entirely, governed by the provider's API terms rather than by our design. A salted hash establishes a commitment to one exact representation of a snapshot; it guarantees neither anonymity nor that the represented statement is true.

5 Grounded Language Understanding and Honest Degradation

Two AI endpoints carry user-visible risk: situation understanding (free text or voice transcript to structured facts and service matches) and document analysis (photo or PDF to extracted fields and risk flags). Both are schema-constrained: generateObject is given a Zod schema and a prompt that embeds the catalogue and forbids inventing services; returned identifiers are joined back to catalogue entries and any identifier not present in the catalogue is dropped before rendering. The user therefore only ever sees official, linkable services. Three properties must stay separate: catalogue validity (the entry is a real official service), recommendation relevance (it suits the resident's situation) and eligibility correctness (the resident qualifies under current rules). The pipeline evidences the first, tests the second preliminarily, and does not establish the third. Where responses carry a confidence value, it is a model-declared score, not a calibrated probability of correctness. Grounding of this kind excludes invented services; it does not guarantee that the best catalogue entry was chosen, and measuring that residual risk is the purpose of Sect. 6.1.

The honest-degradation policy governs failure. If no model key is configured, the endpoint returns 503 instead of a fabricated answer. If the provider call fails, the response is constructed from a clearly labelled SAMPLE object with confidence 0 and a source field set to sample_fallback; the UI renders a visible warning banner, and the issuance path refuses to anchor a credential whose analysis source is sample_fallback. This turns a common demo shortcut, fabricated output that looks identical to real output, into a state that is machine-blocked and human-visible, which we consider a minimum ethical bar for civic AI prototypes.

6 Implementation and Validation

The repository implements 24 application routes (13 statically prerendered, 11 server-rendered), the credential contract, the catalogue as typed data, and an automated test

suite. At submission time the following evidence holds, and Table 2 records what each result does and does not support.

Static and build checks: TypeScript strict type-check passes; ESLint reports zero errors (ten warnings); next build completes for all 24 routes. Functional tests: the Vitest suite passes 12/12, including RLS isolation tests executed against a live PostgreSQL instance that assert one user cannot read or write another user's claims and credentials. Contract checks: local harnesses verify issuer-only issue and revoke, immutability of the stored hash, duplicate-issuance rejection and revocation semantics; a preflight against the Amoy testnet confirms the deployed ABI matches the client (issue(bytes32,bytes32), revoke(bytes32), status(bytes32)), and a read-only bytecode comparison shows the deployed runtime code is identical to the locally compiled source once solc metadata and the deploy-time issuer literal are normalised out; we did not run third-party source verification on a block explorer. Security review remediation: committed credentials were removed from the working tree and HEAD, unauthenticated claim reads were closed (actor() plus per-user filtering on every claims path), and fabricated AI fallbacks were replaced by the labelled-sample policy of Sect. 5. Secrets that had entered git history remain a rotation obligation, tracked as pending on the authors' side.

6.1 Grounded-Matcher Benchmark

To move beyond the qualitative grounding argument of Sect. 5, we exercised the live application endpoint (POST /api/services/match on the built server, same prompt, Zod schema and 28-entry catalogue as the shipped app) over 16 free-text situations. The cases span five input-language groups covering six written varieties: English (11 cases), Hinglish and Roman Urdu (2 cases, grouped as Latin-script mixed transcriptions), Arabic script (1), Devanagari Hindi (1) and Bengali (1). Cases were authored and annotated by one of the authors, each carrying an acceptable service-id set derived from the catalogue's own situation tags: ground truth is reproducible from repository data rather than hand-asserted, but the annotation is single-annotator and the sample is deliberately English-heavy, so per-language counts are reported descriptively and no equal coverage across languages is claimed. Because the labels come from the catalogue's own tags, the benchmark measures agreement with the catalogue, not independent judgement of what is most appropriate for a resident. We report hit@1 (top match acceptable), hit@3 (any returned match acceptable) and the count of out-of-catalogue identifiers in delivered responses. Result (Figure 3): 16/16 hit@1, 16/16 hit@3, 0 out-of-catalogue identifiers delivered, 2.69 catalogue services returned per case (on average 2.38 of them within the acceptable set) and a median end-to-end route latency of 1468 ms (range 1217-1910 ms); latency is measured at the server route and therefore includes the model-provider round trip and network overhead, but excludes client rendering. The delivered out-of-catalogue count is zero by construction, since the route filters non-catalogue identifiers before responding: the result evidences the filter, not the model's hallucination rate on this sample, although all 16 first-attempt responses did parse against the schema. All 16 calls succeeded on the first attempt in the final run; two transient provider 500s observed during earlier development runs (both recovered on retry) are reported here as provider-reliability

noise. Every returned identifier resolved to a real official service, and every top match was checked against its case text. This is consistent with the grounding design of Sect. 5: grounding rules out invented services but not wrong-but-valid catalogue matches. Measuring those requires a larger, independently annotated set including ambiguous, out-of-scope and adversarial inputs (Sect. 7). Re-running the full benchmark against the same build shortly before submission reproduced identical hit@1, hit@3 and out-of-catalogue counts; the two runs differed slightly in aggregate (2.69 vs 2.63 services per case, median of the 16 per-case latencies 1468 vs 1376 ms), which we attribute to provider-side variance, and both raw result files ship with the harness.

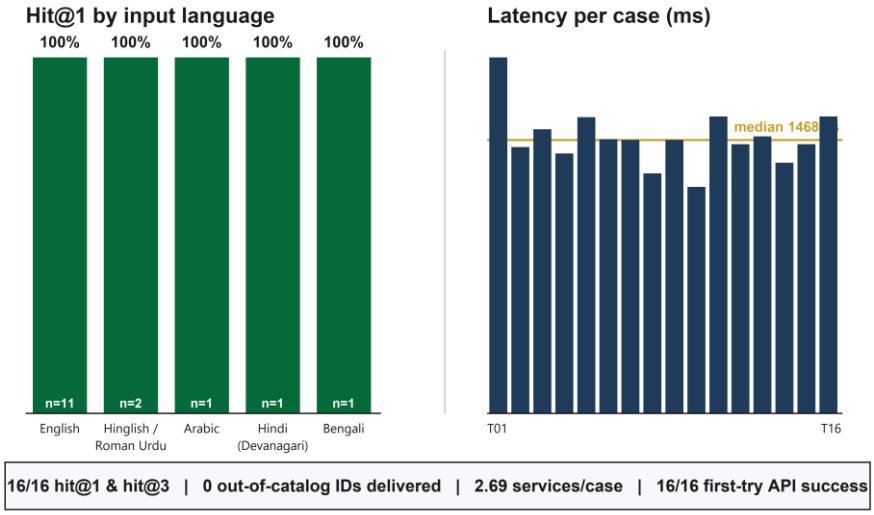


Fig. 3. Grounded-matcher benchmark (16 cases, single annotator). Left: hit@1 by input-language group - English n=11, Hinglish/Roman Urdu n=2, Arabic, Devanagari Hindi and Bengali n=1 each; the sample is not balanced across languages. Right: per-case end-to-end route latency (includes the model-provider round trip) with median line. Footer reports aggregate metrics from the later of the two live endpoint runs.

6.2 Claim Ledger

Table 2. Validation status at submission. Rows marked "not claimed" are reported to bound the paper's claims, not to criticise them.

Mechanism	Evidence at submission	Status
Data isolation (RLS)	12/12 Vitest incl. live-PostgreSQL isolation tests	Verified (prototype)
Registry semantics	Issuer-only, duplicate-reject, one-way revoke, immutable hash	Verified (tests + Amoy ABI preflight)
Catalogue authenticity	28 entries, every URL on u.ae or mohre.gov.ae	URLs verified; freshness unverified
AI grounding (retrieval)	Schema-constrained ids; 16/16 benchmark hit@1; 0 out-of-catalogue delivered (route filter)	Verified on small curated benchmark; user outcomes untested
Honest degradation	503 without key; SAMPLE at confidence 0; issuance blocked	Verified (code + UI)
Factual truth of claims	Chain attests existence/immutability only	Not claimed
Production operation	Testnet only; key rotation pending	Not claimed
Resident usability	No user study conducted	Not claimed

7 Limitations and Ethical Considerations

We state the boundaries plainly. The chain layer runs on a public testnet with a single issuer key; production use requires mainnet deployment, key custody (HSM or multisig), and a revocation policy agreed with the institutions that would honour these anchors. The AI layer's benchmark (Sect. 6.1) is small, single-annotator and scored against catalogue-derived ground truth: it evidences retrieval correctness, not real-world user outcomes, and document extraction quality has still not been measured on real, consented documents. Extending the benchmark with independent annotation, more diverse and ambiguous cases, out-of-scope and adversarial inputs, and a separate consented-document extraction study are the natural next measurement steps. The credential mechanism anchors user-confirmed statements; it adds no independent verification, and an adversary with the issuer key can anchor false statements. The immutable issuer address and public events make such abuse visible and attributable, but they do not prevent it. The demo mode's signed cookie is a hackathon convenience, not an identity system. Finally, the headline demographic figure used in outreach (about 88% expatriate) is indicative and cited as such; all product claims in this paper are bounded by Table 2.

Ethically, the design follows data minimisation by construction: PII stays in user-partitioned RLS tables, nothing personal is written to the chain, and provider-side processing is scoped in Sect. 4.4. The honest-degradation policy exists so that a demonstration can never be mistaken for an operational service, and accessibility (WCAG 2.2, RTL support) and the PDPL-aligned storage model are treated as functional requirements.

8 Conclusion

SANAD shows that one small system can address three barriers residents face even in a world-class e-government: language, evidence clarity and trust. Its core commitments are negative: the model never invents services, the chain never sees personal data, and the product never claims more than its proofs support. The prototype passes its automated validation suite, anchors digests on a public testnet, and ships an explicit proven/not-proven ledger as part of its public surface. We propose, but have not yet arranged, these next steps: consented usability testing with service-seekers, larger independently annotated accuracy benchmarking of the grounded matcher, mainnet custody, and integration of an institutional verifier that can check anchors without access to the off-chain snapshot.

References

[1] United Nations Department of Economic and Social Affairs: United Nations E-Government Survey 2024: Accelerating Digital Transformation for Sustainable Development - With the Addendum on Artificial Intelligence. United Nations, New York (2024). https://doi.org/10.18356/9789211067286

[2] Government of the United Arab Emirates: The UAE in the UN E-Government Development Index. u.ae, https://u.ae/en/about-the-uae/uae-competitiveness/the-uae-in-egovernment-development-index (accessed October 2026)

[3] World Wide Web Consortium: Verifiable Credentials Data Model v2.0. W3C Recommendation, 15 May 2025. https://www.w3.org/TR/vc-data-model/ (2025)

[4] MIT Media Lab and Learning Machine: Blockcerts - The Open Standard for Blockchain Credentials. https://www.blockcerts.org/ (2017)

[5] Bapat, C.: Blockchain for Academic Credentials. arXiv:2006.12665. arXiv (2020). https://arxiv.org/abs/2006.12665

[6] Sheth, R., Sinha, S.R., Patil, M., Beniwal, H., Singh, M.: Beyond Monolingual Assumptions: A Survey on Code-Switched NLP in the Era of Large Language Models across Modalities. In: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 8519-8566. ACL (2026). https://doi.org/10.18653/v1/2026.acl-long.386

[7] United Arab Emirates: Federal Decree by Law No. (45) of 2021 Concerning the Protection of Personal Data. https://uaelegislation.gov.ae/en/legislations/1972 (2021)

[8] World Wide Web Consortium: Web Content Accessibility Guidelines (WCAG) 2.2. W3C Recommendation, 12 December 2024. https://www.w3.org/TR/WCAG22/ (2024)

[9] Nakamoto, S.: Bitcoin: A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf (2008)

[10] UAE Ministry of Human Resources and Emiratisation: Register labour complaints for private-sector employees. https://mohre.gov.ae/en/services/register-labor-complaints-private-sector-employees-2022 (accessed October 2026)

[11] UAE Ministry of Foreign Affairs and International Cooperation: Facts and Figures - Population by Nationality. https://www.mofa.gov.ae/en/the-uae/facts-and-figures (accessed October 2026)